

Trabalho 1 — Servidor HTTP/1.1 sobre sockets TCP

Grupo 7: Luis Felipe Flores Acosta · João Vitor Amaral Rangel · Vinicius Salmoria

Porto Alegre, 2026

Ambiente de teste. Servidor (server.py, Python 3) em 10.132.242.122:8080, --root ./www, bind em 0.0.0.0; cliente remoto em 10.115.242.247 (curl e navegador), ambos em Wi-Fi. Capturas feitas com Wireshark na máquina do servidor (c1.pcapng, c2.pcapng).

1. Arquitetura e concorrência

O servidor usa diretamente a API de sockets (socket, bind, listen, accept, recv, sendall), sem nenhum módulo HTTP. A estrutura tem quatro partes:

- **Laço de aceitação** (main): socket IPv4/TCP em 0.0.0.0, que apenas executa accept() e entrega cada conexão a uma nova thread.
- **Leitura e parsing** (handle_client, parse_request): cada conexão tem um buffer próprio; os bytes de recv() são acumulados até \r\n\r\n, e o que sobra fica no buffer para a próxima requisição. São validados a request line (método, alvo, HTTP/1.1), o formato nome: valor dos cabeçalhos e o Host obrigatório.
- **Resolução segura do caminho** (resolve_requested_path): decodifica o percent-encoding, monta o caminho com os.path.realpath e só o aceita se os.path.commonpath([root, caminho]) == root; caso contrário, 403.
- **Montagem da resposta** (build_response): status line HTTP/1.1, Date (IMF-fixdate/GMT), Server: Grupo-7, Content-Length, Content-Type pela extensão (padrão application/octet-stream) e Connection. O HEAD envia os mesmos cabeçalhos do GET, sem corpo; os outros métodos recebem 405 com Allow: GET, HEAD.

Persistência (Parte 2). Depois de responder, a thread volta ao buffer e espera a próxima requisição no mesmo socket. A conexão só é fechada quando chega Connection: close (repetido na resposta), quando o cliente encerra ou após 5 s sem atividade (settimeout(5)).

Concorrência: uma thread por conexão (threading.Thread, daemon). Escolhemos esse modelo porque o trabalho do servidor é quase todo de E/S bloqueante (recv, send, leitura de disco). Com uma thread por conexão, um cliente lento bloqueia só a sua própria thread, enquanto a principal continua aceitando conexões. Além disso, cada thread guarda naturalmente o estado da sua conexão (buffer, timeout, keep-alive), o que deixa o código simples. I/O não bloqueante (select/asyncio) escalaria melhor para milhares de conexões, mas acrescentaria complexidade que não se justifica neste cenário.

2. Tabela de conformidade

Tabela 1 — Códigos de status obrigatórios (curl executado no servidor, porta 8080)

Código	Requisição curl	Resposta obtida
200 OK	curl.exe -i http://127.0.0.1:8080/index.html	HTTP/1.1 200 OK; Content-Type: text/html; corpo do arquivo
400 Bad Request	curl.exe -i --http1.1 -H "Host:" http://127.0.0.1:8080/	HTTP/1.1 400 Bad Request (HTTP/1.1 sem Host válido)
403 Forbidden	curl.exe -i --path-as-is http://127.0.0.1:8080/../../server.py	HTTP/1.1 403 Forbidden (caminho fora do root)
404 Not Found	curl.exe -i http://127.0.0.1:8080/arquivo-inexistente.txt	HTTP/1.1 404 Not Found
405 Method Not Allowed	curl.exe -i -X POST http://127.0.0.1:8080/index.html	HTTP/1.1 405 Method Not Allowed + Allow: GET, HEAD

Todas as respostas, inclusive as de erro, trazem Content-Length correto, Date, Server: Grupo-7 e Content-Type.

3. Demonstração de segurança (travessia de diretório)

Usamos --path-as-is para que o curl envie o caminho sem normalizá-lo. O alvo das três tentativas é o server.py, que fica fora de www/.

Tabela 2 — Tentativas de travessia

Técnica	Requisição enviada	Resposta
"../" literal	curl.exe -v --path-as-is http://127.0.0.1:8080/../../server.py	HTTP/1.1 403 Forbidden
Pontos codificados (%2E%2E)	curl.exe -v --path-as-is http://127.0.0.1:8080/%2E%2E/%2E%2E/server.py	HTTP/1.1 403 Forbidden
Barras codificadas (%2F)	curl.exe -v --path-as-is http://127.0.0.1:8080/..%2F..%2Fserver.py	HTTP/1.1 403 Forbidden

```
PS C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores> python server.py
--port 8080 --root ./www
Servidor HTTP/1.1 iniciado
Endereço: 0.0.0.0:8080
Root: C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores\www
[127.0.0.1:64024] conexão aberta
[127.0.0.1:64024] GET ../../server.py -> 403 Forbidden
[127.0.0.1:64024] conexão encerrada
[127.0.0.1:64028] conexão aberta
[127.0.0.1:64028] GET /%2E%2E/%2E%2E/server.py -> 403 Forbidden
[127.0.0.1:64028] conexão encerrada
[127.0.0.1:64031] conexão aberta
[127.0.0.1:64031] GET /..%2F..%2Fserver.py -> 403 Forbidden
[127.0.0.1:64031] conexão encerrada
```

Figura 1 — Log do servidor: as três tentativas recebem 403 Forbidden.

A validação acontece **depois** da decodificação e sobre o caminho real resolvido. Por isso, as variantes codificadas são barradas da mesma forma que o "../" literal, e o conteúdo de server.py não é exposto em nenhum dos casos.

4. Captura de uma transação completa

Requisição feita pelo cliente remoto: curl.exe -v -H "Connection: close"

http://10.132.242.122:8080/index.html. Usamos Connection: close para que a mesma captura mostrasse também o encerramento da conexão.

No.	Time	Source	Destination	Protocol	Length	Info
1523	37.330961500	10.115.242.247	10.132.242.122	TCP	54	60771 → 8080 [ACK] Seq=310 Ack=287 Win=65024 Len=0
1524	37.330963100	10.115.242.247	10.132.242.122	TCP	54	60771 → 8080 [FIN, ACK] Seq=310 Ack=287 Win=65024 Len=0
1525	37.331056500	10.132.242.122	10.115.242.247	TCP	54	8080 → 60771 [ACK] Seq=287 Ack=311 Win=65024 Len=0
1526	37.335111700	10.132.242.122	10.115.242.247	TCP	54	8080 → 60770 [FIN, ACK] Seq=3679986 Ack=1107 Win=64256 Len=0
1527	37.345780300	10.115.242.247	10.132.242.122	TCP	54	60770 → 8080 [ACK] Seq=1107 Ack=3679987 Win=1048320 Len=0
1528	37.345781000	10.115.242.247	10.132.242.122	TCP	54	60770 → 8080 [FIN, ACK] Seq=1107 Ack=3679987 Win=1048320 Len=0
1529	37.345837300	10.132.242.122	10.115.242.247	TCP	54	8080 → 60770 [ACK] Seq=3679987 Ack=1108 Win=64256 Len=0
1539	38.918975200	10.115.242.247	10.132.242.122	TCP	66	60811 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1540	38.919162700	10.132.242.122	10.115.242.247	TCP	66	8080 → 60811 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1541	38.927655900	10.115.242.247	10.132.242.122	TCP	54	60811 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1542	38.954638100	10.115.242.247	10.132.242.122	HTTP	166	GET /index.html HTTP/1.1
1543	38.956210800	10.132.242.122	10.115.242.247	HTTP	826	HTTP/1.1 200 OK (text/html)
1544	38.956587500	10.132.242.122	10.115.242.247	TCP	54	8080 → 60811 [FIN, ACK] Seq=773 Ack=113 Win=65280 Len=0
1545	38.966349100	10.115.242.247	10.132.242.122	TCP	54	60811 → 8080 [ACK] Seq=113 Ack=774 Win=64512 Len=0
1546	39.037536000	10.115.242.247	10.132.242.122	TCP	54	60811 → 8080 [FIN, ACK] Seq=113 Ack=774 Win=64512 Len=0
1547	39.037643900	10.132.242.122	10.115.242.247	TCP	54	8080 → 60811 [ACK] Seq=774 Ack=114 Win=65280 Len=0

Figura 2 — GET /index.html de 10.115.242.247 para 10.132.242.122 (pacotes 1539–1547).

Etapa	Pacotes	Conteúdo
Handshake TCP	1539, 1540, 1541	SYN → SYN-ACK → ACK (porta cliente 60811 ↔ 8080)
Requisição	1542 (166 B)	GET /index.html HTTP/1.1 + Host + Connection: close
Resposta	1543 (826 B)	HTTP/1.1 200 OK (text/html); cabeçalhos e corpo em um único segmento
Encerramento	1544–1547	Servidor FIN-ACK → cliente ACK → cliente FIN-ACK → servidor ACK

O servidor é quem envia o primeiro FIN (1544), logo depois da resposta, como esperado quando o cliente pede Connection: close.

5. Evidência de atendimento simultâneo

A máquina 10.115.242.247 abriu uma conexão (porta 61006) e passou a enviar os cabeçalhos aos poucos, deixando a requisição incompleta. Enquanto isso, uma segunda máquina (10.132.242.122, porta 56353) pediu /index.html.

```
[10.115.242.247:60985] conexão aberta
[10.115.242.247:60985] GET /index.html -> 200 OK
[10.115.242.247:60985] conexão encerrada
[10.132.242.122:63930] conexão aberta
[10.132.242.122:63930] GET /index.html -> 200 OK
[10.132.242.122:63930] conexão encerrada
[10.115.242.247:61006] conexão aberta
[10.132.242.122:56353] conexão aberta
[10.132.242.122:56353] GET /index.html -> 200 OK
[10.132.242.122:56353] conexão encerrada
```

Figura 3 — Log do servidor: a conexão 56353 é aberta, atendida (200 OK) e encerrada enquanto a 61006 continua aberta.

A segunda requisição foi concluída sem esperar a primeira, ou seja, a conexão lenta não bloqueou as demais. É o comportamento esperado do modelo de uma thread por conexão.

6. RTT medido

O ping não funcionou porque o ICMP estava bloqueado na rede. Por isso, o RTT foi medido nas próprias capturas. Como o Wireshark rodava na máquina do servidor, o RTT corresponde ao intervalo entre um segmento enviado pelo servidor e a resposta imediata do cliente: **SYN-ACK** → **ACK** no handshake e **FIN** → **ACK** no encerramento. O intervalo SYN → SYN-ACK, medido no servidor, não serve para isso: ele vale apenas ≈ 0,2 ms e reflete só o tempo de processamento local da pilha TCP.

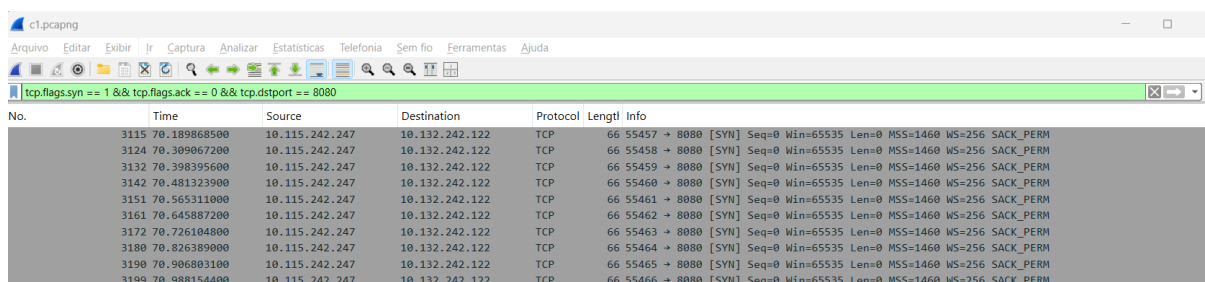
Tabela 3 — Amostras de RTT

Captura / pacotes	Intervalo	RTT
Transação (1540 → 1541)	SYN-ACK 38,919163 → ACK 38,927656 s	8,49 ms
Transação (1544 → 1545)	FIN 38,956588 → ACK 38,966349 s	9,76 ms
C2 (76 → 77)	SYN-ACK 6,295915 → ACK 6,313080 s	17,17 ms
C2 (101 → 102)	FIN 6,462824 → ACK 6,470132 s	7,31 ms
Média		≈ 10,7 ms

A variação (7–17 ms) é típica de Wi-Fi. A média de ≈ 10,7 ms é o valor de referência usado na Seção 9.

7. Comparação C1 × C2

Foram feitas 10 requisições sequenciais a /teste.txt. Em **C1**, cada uma usou uma conexão nova com Connection: close; em **C2**, todas passaram por uma única conexão persistente. Os valores vêm das propriedades das capturas, filtradas pelo tráfego cliente↔servidor na porta 8080.



No.	Time	Source	Destination	Protocol	Length	Info
3115	70.189868500	10.115.242.247	10.132.242.122	TCP	66	55457 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3124	70.309067200	10.115.242.247	10.132.242.122	TCP	66	55458 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3132	70.398395600	10.115.242.247	10.132.242.122	TCP	66	55459 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3142	70.481323900	10.115.242.247	10.132.242.122	TCP	66	55460 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3151	70.565311000	10.115.242.247	10.132.242.122	TCP	66	55461 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3161	70.645887200	10.115.242.247	10.132.242.122	TCP	66	55462 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3172	70.726104800	10.115.242.247	10.132.242.122	TCP	66	55463 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3180	70.826389000	10.115.242.247	10.132.242.122	TCP	66	55464 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3190	70.906803100	10.115.242.247	10.132.242.122	TCP	66	55465 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3199	70.988154400	10.115.242.247	10.132.242.122	TCP	66	55466 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

Figura 4 — C1: 10 segmentos SYN (portas 55457–55466), ou seja, 10 conexões.

No.	Time	Source	Destination	Protocol	Length	Info
75	6.295599800	10.115.242.247	10.132.242.122	TCP	66	55516 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
76	6.295914600	10.132.242.122	10.115.242.247	TCP	66	8080 → 55516 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
77	6.313080100	10.115.242.247	10.132.242.122	TCP	54	55516 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
78	6.313082400	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
79	6.317078900	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
80	6.330301400	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
81	6.331687000	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
83	6.345778000	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
84	6.347842300	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
85	6.360186600	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
86	6.362036500	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
87	6.370402900	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
88	6.372853300	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
89	6.391413100	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
90	6.392406000	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
91	6.411084900	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
92	6.413069400	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
93	6.426658000	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
94	6.427738900	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
95	6.441014300	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
96	6.442447800	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
97	6.451381100	10.115.242.247	10.132.242.122	HTTP	146	GET /teste.txt HTTP/1.1
98	6.452518900	10.132.242.122	10.115.242.247	HTTP	252	HTTP/1.1 200 OK (text/plain)
99	6.462466800	10.115.242.247	10.132.242.122	TCP	54	55516 → 8080 [FIN, ACK] Seq=921 Ack=1981 Win=65024 Len=0
100	6.462634300	10.132.242.122	10.115.242.247	TCP	54	8080 → 55516 [ACK] Seq=1981 Ack=922 Win=64512 Len=0
101	6.462823600	10.132.242.122	10.115.242.247	TCP	54	8080 → 55516 [FIN, ACK] Seq=1981 Ack=922 Win=64512 Len=0
102	6.470132200	10.115.242.247	10.132.242.122	TCP	54	55516 → 8080 [ACK] Seq=922 Ack=1982 Win=65024 Len=0

Figura 5 — C2: um handshake (75–77), 10 pares GET/200 OK (78–98) e um encerramento (99–102).

Tabela 4 — Métricas de C1 e C2

Métrica	C1 (nova conexão)	C2 (persistente)	Economia com C2
Handshakes TCP completos	10	1	9 a menos (90%)
Total de pacotes	96	27	71,9%
Bytes totais	8.464 B	4.382 B	48,2%
Tempo total	1,229 s	0,175 s	85,8%

Estatísticas

Medição	Capturado(s)	Exibidos	Marcado(s)
Pacotes	4512	96 (2.1%)	—
Intervalo de tempo, s	119.649	1.229	—
Média de pps	37.7	78.1	—
Tamanho médio do pacote, B	822	88	—
Bytes	3708502	8464 (0.2%)	0
Média de bytes/s	30 k	6885	—
Média de bits/s	247 k	55 k	—

Figura 6 — Propriedades da captura C1 (exibidos): 96 pacotes, 8.464 bytes, 1,229 s.

Estatísticas

Medição	Capturado(s)	Exibidos	Marcado(s)
Pacotes	171	27 (15.8%)	—
Intervalo de tempo, s	14.325	0.175	—
Média de pps	11.9	154.7	—
Tamanho médio do pacote, B	356	162	—
Bytes	60874	4382 (7.2%)	0
Média de bytes/s	4249	25 k	—
Média de bits/s	33 k	200 k	—

Figura 7 — Propriedades da captura C2 (exibidos): 27 pacotes, 4.382 bytes, 0,175 s.

Economia de pacotes = $(96 - 27) / 96 = 71,9\%$. Economia de bytes = $(8.464 - 4.382) / 8.464 = 48,2\%$.

8. Overhead de conexão em C1

Cada conexão de C1 gasta 3 pacotes na abertura (SYN 66 B + SYN-ACK 66 B + ACK 54 B = 186 B) e 4 no fechamento (2 FIN + 2 ACK, 4×54 B = 216 B), ou seja, **7 pacotes e 402 bytes só de controle**.

Tabela 5 — Custo de abrir e fechar conexões

	Pacotes	Bytes	% do total de C1
Abertura (10 handshakes)	30	1.860	—
Fechamento (10 encerramentos)	40	2.160	—
Total abrir + fechar em C1	70	4.020	72,9% dos pacotes; 47,5% dos bytes
Mesmo custo em C2 (1 conexão)	7	402	—

Isso explica quase toda a diferença entre os cenários. Dos 69 pacotes a mais em C1, 63 vêm das 9 conexões extras (9×7) e 6 são ACKs puros adicionais (6×54 B). Nos bytes, 3.618 B vêm das conexões extras, 324 B desses ACKs e 140 B da diferença nos cabeçalhos (+19 B de Connection: close em cada

GET e -5 B em cada resposta, close × keep-alive): $3.618 + 324 + 140 = 4.082$ B, que é exatamente a diferença $8.464 - 4.382$. O filtro `tcp.port == 8080 && ip.addr == 10.115.242.247 && tcp.len == 0` em C1 confirma o resultado (Figura 8): 76 pacotes e 4.344 B sem payload, isto é, os 70 de abrir/fechar mais os 6 ACKs.

Estatísticas			
Medição	Capturado(s)	Exibidos	Marcado(s)
Pacotes	4512	76 (1.7%)	—
Intervalo de tempo, s	119.649	1.229	—
Média de pps	37.7	61.8	—
Tamanho médio do pacote, B	822	57	—
Bytes	3708502	4344 (0.1%)	0
Média de bytes/s	30 k	3533	—
Média de bits/s	247 k	28 k	—

Figura 8 — C1 filtrado por `tcp.len == 0`: 76 pacotes, 4.344 bytes de controle TCP.

9. Análise em função do RTT

Pelo modelo do Kurose & Ross, uma conexão nova custa **1 RTT de handshake** (o cliente só envia o GET depois do SYN-ACK) mais **1 RTT** de requisição/resposta. Uma conexão já aberta custa só o segundo. O FIN não atrasa a próxima requisição, então não entra na conta. Para $N = 10$:

$$C1 \approx N \times (1 \text{ RTT handshake} + 1 \text{ RTT req/resp}) = 10 \times 2 = 20 \text{ RTT}$$

$$C2 \approx 1 \text{ RTT handshake} + N \times 1 \text{ RTT req/resp} = 1 + 10 = 11 \text{ RTT}$$

$$\text{Diferença} \approx (N - 1) \times \text{RTT} = 9 \text{ RTT} \approx 9 \times 10,7 \text{ ms} \approx 96 \text{ ms}$$

Os **9 RTTs a mais vêm inteiramente dos 9 handshakes extras**: cada conexão nova obriga o cliente a esperar a ida do SYN e a volta do SYN-ACK antes de enviar o GET. A diferença medida, porém, foi de $1,229 - 0,175 = 1,054$ s, e os 9 RTTs explicam só cerca de 9% dela. O restante é custo fixo por conexão, que independe da rede. Na captura de C1, os SYNs aparecem a cada ≈ 89 ms, enquanto em C2 os GETs chegam a cada ≈ 15 ms. Esse intervalo maior em C1 é a criação e o fechamento de sockets e, principalmente, o disparo de uma nova execução do cliente para cada requisição. Com um RTT de ~ 10 ms em rede local, esse custo de software pesa mais que a propagação.

10. Conclusão

A conexão persistente reduziu os pacotes em 71,9%, os bytes em 48,2% e o tempo em 85,8%, porque elimina 9 dos 10 handshakes e encerramentos. A parte do ganho que depende da rede é $(N - 1) \times \text{RTT}$: ela cresce com o número de conexões evitadas e com a latência entre cliente e servidor. Neste experimento, o RTT era baixo ($\approx 10,7$ ms), por isso o ganho atribuível à rede foi de apenas ~ 96 ms.

A vantagem seria ainda maior em uma rede de **alto RTT com muitos objetos por página**: cliente e servidor distantes geograficamente, redes móveis ou enlaces de satélite. Com RTT de 100 ms, os mesmos 10 pedidos economizariam $9 \times 100 \text{ ms} = 0,9$ s. Com RTT de 600 ms (satélite), 5,4 s. Uma página com 50 recursos pagaria 49 handshakes a mais. Nesses casos o custo de rede passa a dominar o custo fixo de software, e a conexão persistente se torna decisiva. A vantagem aumenta ainda mais com TLS, que acrescenta RTTs ao estabelecimento de cada conexão, e com o slow start do TCP, que recomeça a cada conexão nova.